OPEN-SOURCE R WORKFLOWS FOR MODERN GEOSPATIAL AND ACCESSIBILITY ANALYSIS

Ari Tilles

California State University, Northridge





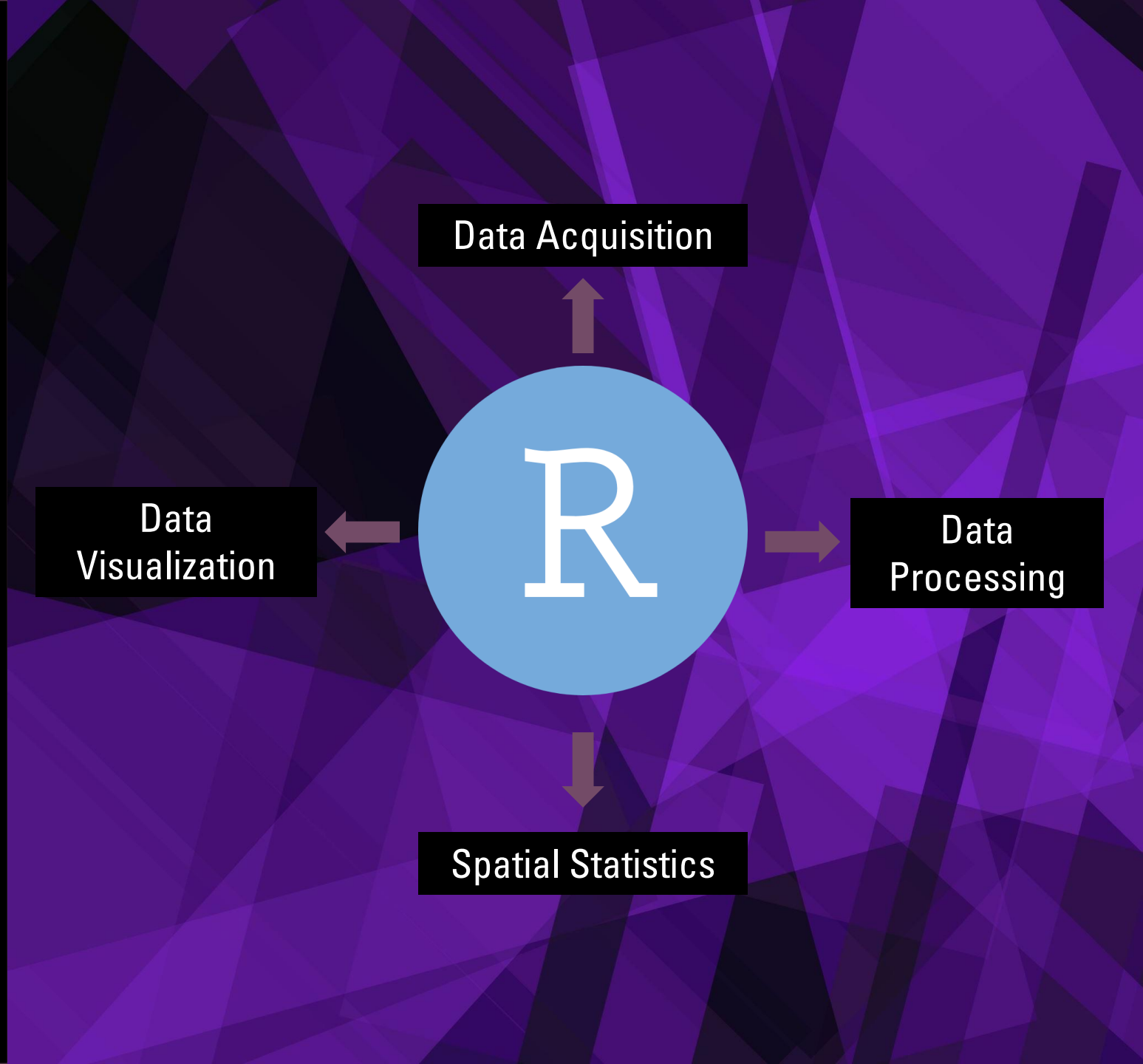
ABOUT ME

- M.S. Geographic Information Science, CSU Northridge (expected 2027)
- B.A. Statistics, UC Davis (2025)
- Research interests: spatial data science, accessibility, urban & transportation planning
- I enjoy geography, travel, sports, and music.
- How I got into GIS and R…



WHY R FOR GIS?

- Versatility
- Customization
- Documentation
- Reproducibility



OVERVIEW

- Workflow showcasing how R can be used for various GIS processes
- Accessibility study to hospitals within Sacramento Metropolitan Area
- Study area: Sacramento County
- Hospital Data from Centers for Medicare and Medicaid Services (CMS)



DATA ACQUISITION

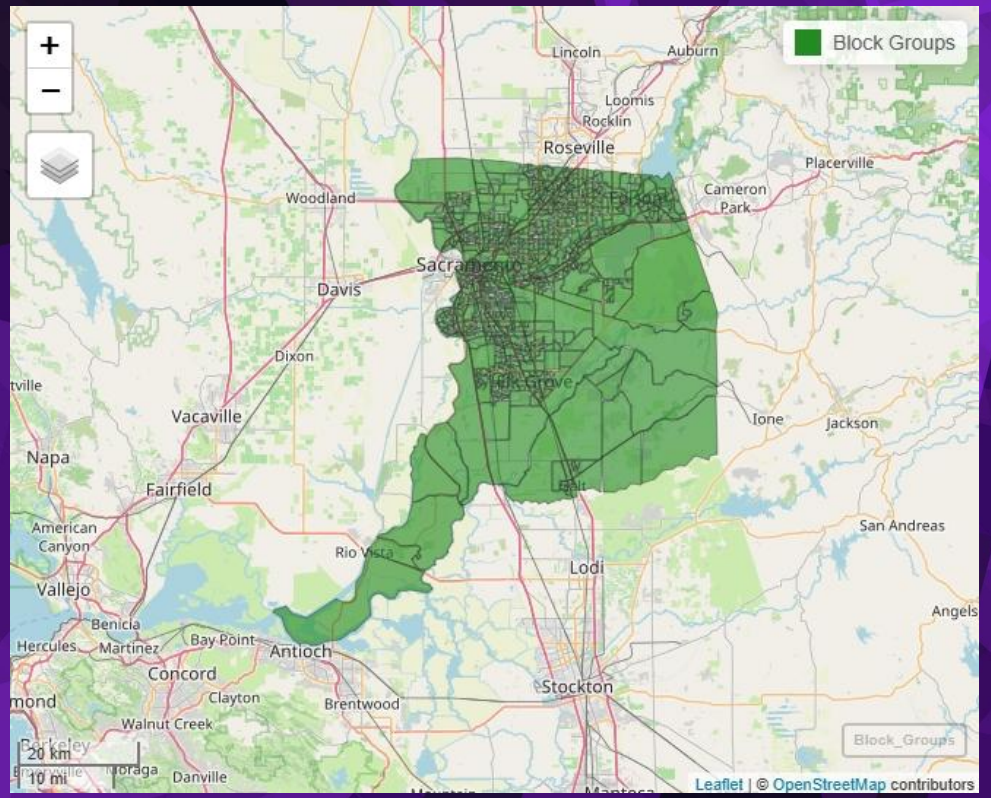
BOUNDARY DATA

TIGRIS

- Great for U.S. boundary data from census
- Can be used to directly download TIGER/Line shapefiles
- Can download for different geographic units like states, counties, census tracts, block groups, roads, etc.
- Various years are available to download data for

```r
#Importing library
library(tigris)

#Importing block groups for our study area
block_groups <- block_groups(state = "California",
                             county = "Sacramento", year = 2024)
```



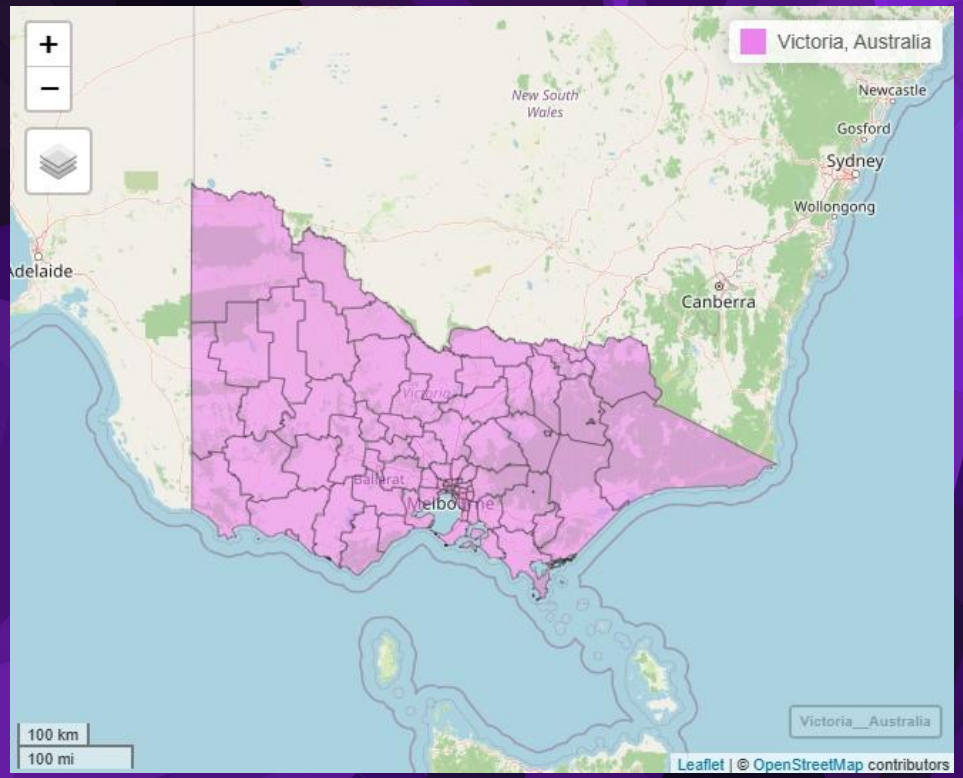
GADM

- Part of the geodata package
- Used to download administrative boundaries for all countries and their subdivisions
- Sidenote: geodata can be used to download other useful data related to crops, landcover, population density, soil, climate, etc.

```r
#Importing library
library(geodata)

# Download GADM level 2 (Counties) for Australia
australia_gadm <- gadm(country = "AUS", level = 2, path = tempdir())

# Subsetting State of Victoria
victoria <- australia_gadm[australia_gadm$NAME_1 == "Victoria", ]
```



CENSUS DATA

TIDYCENSUS

- Allows users to download U.S Census data directly using an API
  - Requires a free API key
- Can download data from Decennial Census, ACS, etc.
- GEOIDs come out clean
- Easy to combine with dplyr to automatically remove margin of error columns
- Includes other useful functions to find variables, interpolate between geographies and get FIPS codes

```r
#Importing libraries
library(tidycensus)
library(dplyr)

#Downloading total population for our study area
total_pop <- get_acs(geography = "block group",
                     table = "B01003", #You can also select specific
                                       #variables if you know them
                                       #(e.g. B01003_01)
                     year = 2024,
                     state = "California",
                     county = "Sacramento",
                     output = "wide",
                     geometry = FALSE) %>%
  select(-ends_with("M"))  #dplyr function to remove margin of error columns
```

	GEOID	NAME	B01003_001E
1	060670001001	Block Group 1; Census Tract 1; Sacramento County; California	2128
2	060670001002	Block Group 2; Census Tract 1; Sacramento County; California	1115
3	060670001003	Block Group 3; Census Tract 1; Sacramento County; California	570
4	060670002001	Block Group 1; Census Tract 2; Sacramento County; California	659
5	060670002002	Block Group 2; Census Tract 2; Sacramento County; California	1133
6	060670002003	Block Group 3; Census Tract 2; Sacramento County; California	800
7	060670002004	Block Group 4; Census Tract 2; Sacramento County; California	1309
8	060670003001	Block Group 1; Census Tract 3; Sacramento County; California	923
9	060670003002	Block Group 2; Census Tract 3; Sacramento County; California	1056
10	060670003003	Block Group 3; Census Tract 3; Sacramento County; California	1450
11	060670004001	Block Group 1; Census Tract 4; Sacramento County; California	854
12	060670004002	Block Group 2; Census Tract 4; Sacramento County; California	470
13	060670004003	Block Group 3; Census Tract 4; Sacramento County; California	978
14	060670004004	Block Group 4; Census Tract 4; Sacramento County; California	677
15	060670004005	Block Group 5; Census Tract 4; Sacramento County; California	994
16	060670005011	Block Group 1; Census Tract 5.01; Sacramento County; California	793
17	060670005012	Block Group 2; Census Tract 5.01; Sacramento County; California	1044
18	060670005021	Block Group 1; Census Tract 5.02; Sacramento County; California	1039
19	060670005022	Block Group 2; Census Tract 5.02; Sacramento County; California	644

GEOCODING

TIDYGEOCODER

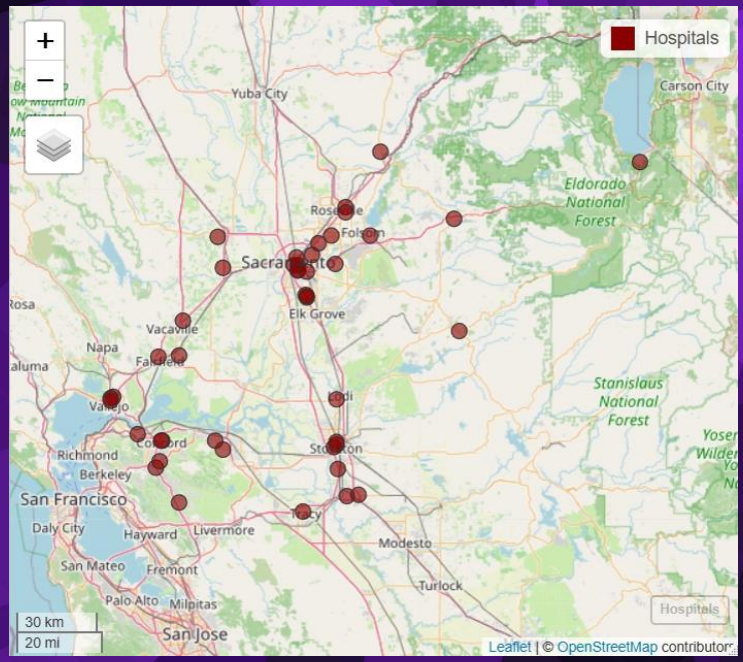
- Provides single and batch geocoding through a variety of services
  - OSM, Census, ArcGIS, etc.
- Can perform geocoding or reverse geocoding
- Useful to combine multiple geocoding services to minimize null results

```r
#Importing library
library(tidygeocoder)

hospital_coords_osm <- geo(street = hospitals$Address,
                      city = hospitals$City.Town,
                      state = hospitals$State,
                      postalcode = hospitals$ZIP.Code,
                      method = "osm") #Can use different geocoding services
                                      #such as OSM or the Census

hospital_coords_census <- geo(street = hospitals$Address,
                      city = hospitals$City.Town,
                      state = hospitals$State,
                      postalcode = hospitals$ZIP.Code,
                      method = "census")

hospital_coords <- left_join(
  hospital_coords_osm,
  hospital_coords_census,
  by = "street"
) %>%
  mutate(
    city = coalesce(city.x, city.y),
    state = coalesce(state.x, state.y),
    postalcode = coalesce(postalcode.x, postalcode.y),
    lat = coalesce(lat.x, lat.y),
    long = coalesce(long.x, long.y)
  ) %>%
  select(street, city, state, postalcode, lat, long)
```



GEOPROCESSING

SF

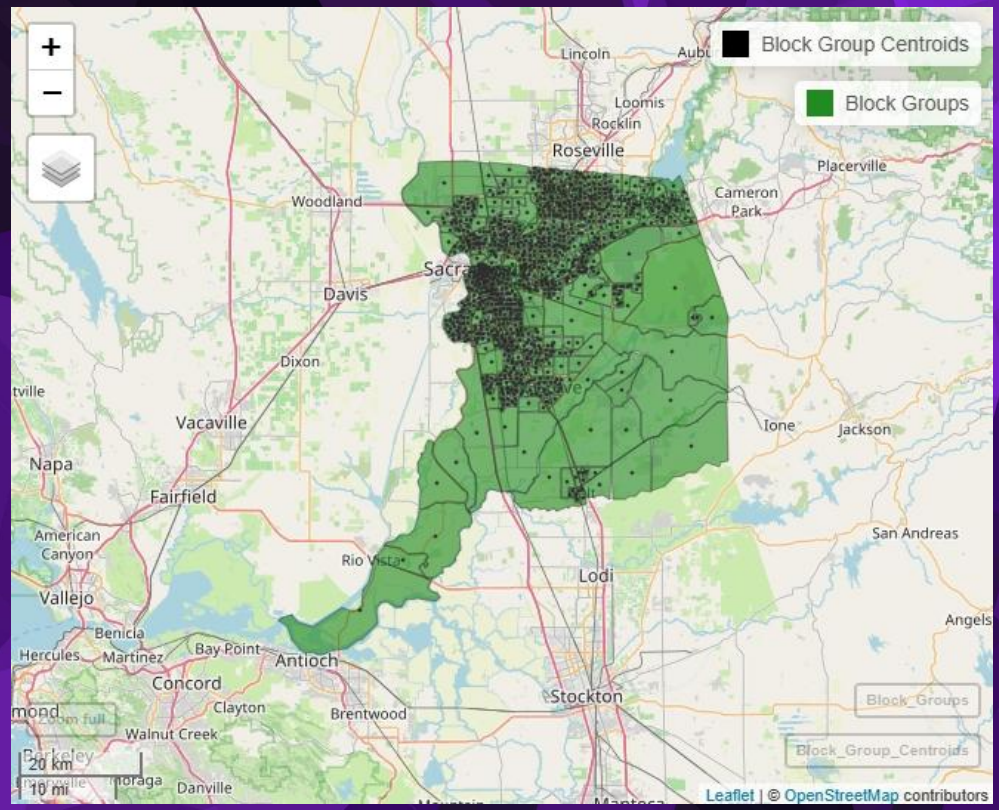
- Allows users to access GDAL library
- Backbone of geoprocessing in R
- Many of the functions are very similar if not the same to PostGIS
- Too many useful functions to mention!

```r
#Importing library
library(sf)

#Creating centroids from block groups
bg_centroids <- st_centroid(block_groups)

#Transforming the centroids to a chosen projection
bg_centroids <- st_transform(bg_centroids, 4326)

#Extracting lat/lon from block group centroids from geometry
coords <- st_coordinates(bg_centroids)
```



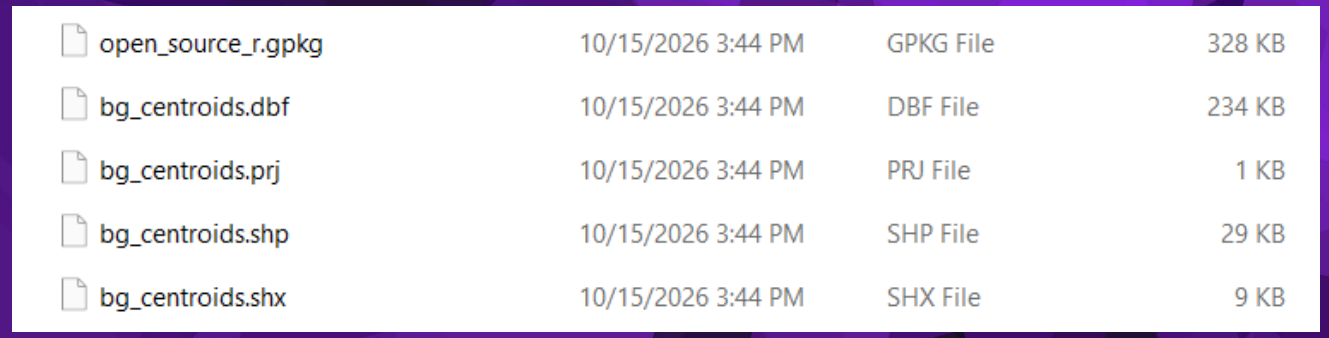
SF

- SF is the mechanism to read spatial data into R through st_read() and export it out through st_write()
- These functions allow for the reading/ writing of many different geospatial data formats
- Examples: ShapeFile, GeoPackage, GeoJSON, KML/KMZ, CSV, etc.

```r
#Exporting to Shapefile and GPKG
st_write(bg_centroids, "bg_centroids.shp", append = FALSE)

st_write(bg_centroids, dsn = "open_source_r.gpkg",
         layer = "bg_centroids",append = FALSE)
st_write(hospital_points, dsn = "open_source_r.gpkg",
         layer = "hospital_points", append = FALSE)

#Reading in data from GPKG
bg_centroids_read <- st_read(dsn = "open_source_r.gpkg", layer = "bg_centroids")
```



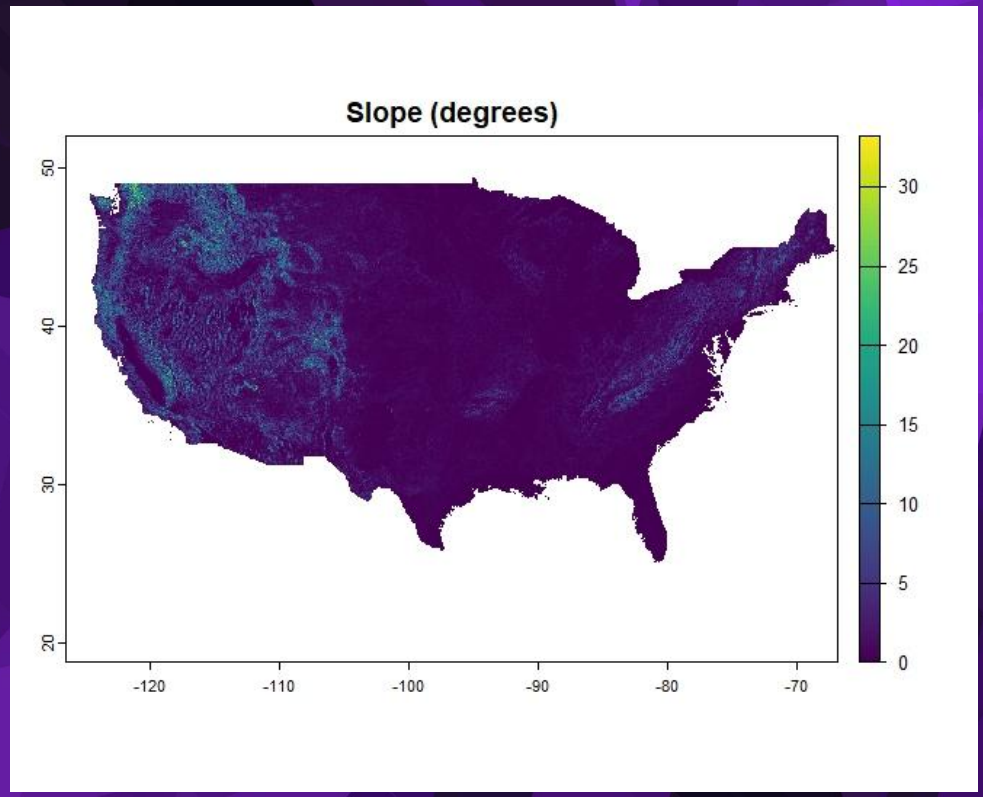
TERRA

- A premier package for working with raster data
- Also provides access to the GDAL library

```r
#Importing libraries
library(terra)
library(geodata)

# Download a global DEM tile (SRTM)
dem <- elevation_30s(country = "United States", path = tempdir())

#Calculating and visualizing slope using terra
slope <- terrain(dem, v = "slope", unit = "degrees")
plot(slope, main = "Slope (degrees)")
```



ROUTING

R5R

- An open-source multimodal routing package based on the Conveyal R5 routing engine
- R5r requires a .pbf road network file to create network
- Can optionally include GTFS .zip files and .tif DEM files
- Allows for routing via a variety of transit modes: Bus, train, car, bicycle, walking, etc.

```r
#Allocating memory and importing library
options(java.parameters = '-Xmx12G') #"12G" refers to 12 GB of memory
library(r5r)

#Building transport network from pbf file
r5r_net <- build_network(
  data_path = "Data"  #Folder in my directory where the pbf file is found
)

#The function reads for a pbf file in the directory
#R5r also takes GTFS feeds as .zip files and .tif DEMs if in the directory

#Origins
origins <- bg_centroids %>%
  select(id = GEOID, lat, lon = long)

#Destinations
destinations <- hospital_points %>%
  select(id = hospital_id, lat, lon = long)

#Setting departure time (more relevant for routing with GTFS)
#If no departure time is set, r5r uses current system time
departure_time = as.POSIXct("2026-10-05 08:00:00")
```

R5R – TRAVEL TIME MATRICES

- Produces Origin-Destination matrices using previously built network, origin and destination inputs
- R5r computes paths based on shortest travel time, not shortest travel distance
- High levels of customization!
- Examples: Modes, Max trip duration, Walking/biking speed, Max number of segments or fares, etc.

```
ttm <- travel_time_matrix(
  r5r_net,
  origins = origins,
  destinations = destinations,
  mode = "CAR",
  departure_datetime = departure_time,
  max_trip_duration = 45,    # minutes
)
```

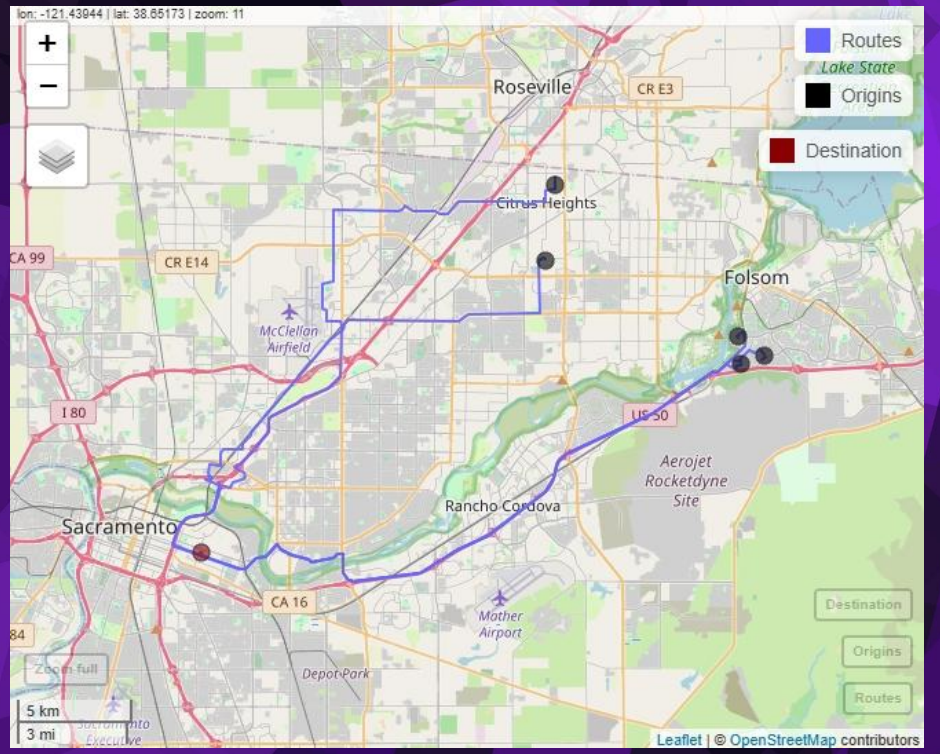
	from_id	to_id	travel_time_p50
1	060670085041	2	24
2	060670085041	7	23
3	060670085041	14	31
4	060670085041	16	19
5	060670085041	17	15
6	060670085041	22	7
7	060670085041	23	22
8	060670085041	25	38
9	060670085041	26	18
10	060670085041	28	41
11	060670085041	29	30
12	060670085041	30	23
13	060670085041	31	30
14	060670085041	36	19
15	060670085041	37	23
16	060670085041	39	29
17	060670085041	40	20
18	060670085041	41	23
19	060670085041	44	28

R5R – DETAILED ITINERARIES

- Provides detailed travel itineraries including times and distances for all segments
- Same customization options as for creating travel time matrices
- Creates and allows for viewing of route geometries

```
#Detailed itinerary example
itins_subset <- detailed_itineraries(
  r5r_net,
  origins = origins[1:5,], # A subset of the first five origins
  destinations = destinations[1:5,],
  mode = "CAR",
  departure_datetime = departure_time,
  max_trip_duration = 45,
  all_to_all = TRUE,
)
```

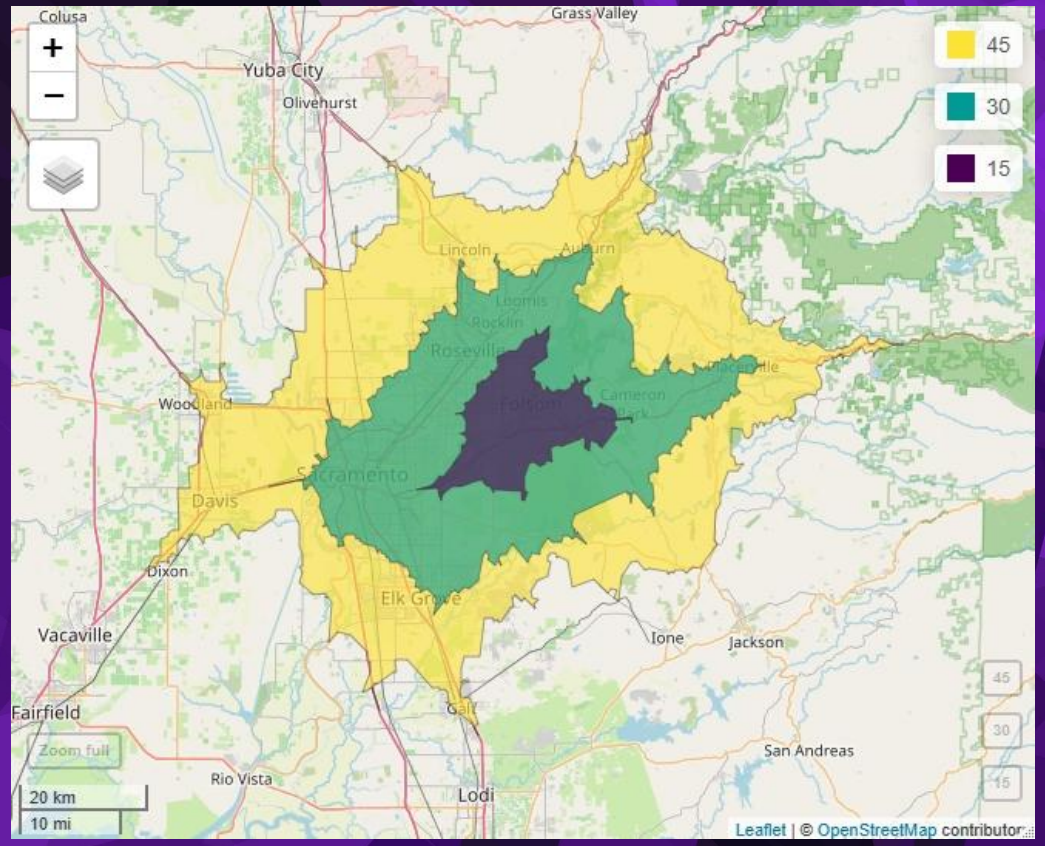
	from_id	from_lat	from_lon	to_id	to_lat	to_lon	option	departure_time	total_duration	total_distance
1	060670085041	38.65472	-121.1856	2	38.57057	-121.453	1	08:00:00	28.8	30386
2	060670085042	38.64750	-121.1721	2	38.57057	-121.453	1	08:00:00	30.9	30649
3	060670085043	38.64425	-121.1840	2	38.57057	-121.453	1	08:00:00	30.1	29270
4	060670081413	38.68467	-121.2816	2	38.57057	-121.453	1	08:00:00	35.5	27426
5	060670081431	38.71402	-121.2765	2	38.57057	-121.453	1	08:00:00	44.6	33797



R5R - ISOCHRONES

- Allows for easy visualization of travel time isochrones
- Many of the same customization options available as when creating travel time matrices
- Different time cutoffs allowed

```r
#Isochrone function
iso <- isochrone(
  r5r_net,
  origins = origins[1,],   # A subset of one origin
  mode = "CAR",
  departure_datetime = departure_time,
  cutoffs = c(15, 30, 45)
)
```



R5R - ACCESSIBILITY

- A function that allows for the calculation accessibility to opportunities
- Similar customization options as previous functions and includes ability to include decay function
- There is also a more detailed Accessibility package with more options

```
access <- accessibility(
  r5r_net ,
  origins = origins,
  destinations = destinations,
  opportunities_colnames = "hospital_count",
  mode = "CAR",
  departure_datetime = departure_time,
  cutoffs = c(10,20,30),
  max_trip_duration = 30
)
```

	id	opportunity	percentile	cutoff	accessibility
1	060670085041	hospital_count	50	10	1
2	060670085041	hospital_count	50	20	5
3	060670085041	hospital_count	50	30	14
4	060670085042	hospital_count	50	10	1
5	060670085042	hospital_count	50	20	2
6	060670085042	hospital_count	50	30	12
7	060670085043	hospital_count	50	10	1
8	060670085043	hospital_count	50	20	4
9	060670085043	hospital_count	50	30	15
10	060670081413	hospital_count	50	10	1
11	060670081413	hospital_count	50	20	6
12	060670081413	hospital_count	50	30	15
13	060670081431	hospital_count	50	10	1
14	060670081431	hospital_count	50	20	6
15	060670081431	hospital_count	50	30	17
16	060670081441	hospital_count	50	10	0
17	060670081441	hospital_count	50	20	5
18	060670081441	hospital_count	50	30	13

SPATIAL STATISTICS

SPDEP

- A spatial statistics package useful for creating spatial weights matrices and testing for spatial autocorrelation
- Useful for calculating statistics such as Moran's I and Geary's C
- Other packages help with other spatial statistics
  - Example: "spatialreg" for spatial regression

```
#Importing library
library(spdep)
#Testing for spatial autocorrelation among accessibility values

#Build contiguity neighbors (Queen)
#Create polygon neighbors with poly2nb():
bg_nb <- poly2nb(block_groups) # Queen contiguity (default)
```

```
#Convert neighbors → weights list
#Many tests in spdep use a listw object:
bgweightsw <- nb2listw(bg_nb, style = "W")  # row-standardized

#Moran's I of Accessible Hospitals
bg_moran <- moran.test(block_groups$accessibility, bgweightsw,
                       zero.policy = TRUE, na.action = na.exclude)
```

```
        Moran I test under randomisation

data:  block_groups$accessibility
weights: bgweightsw

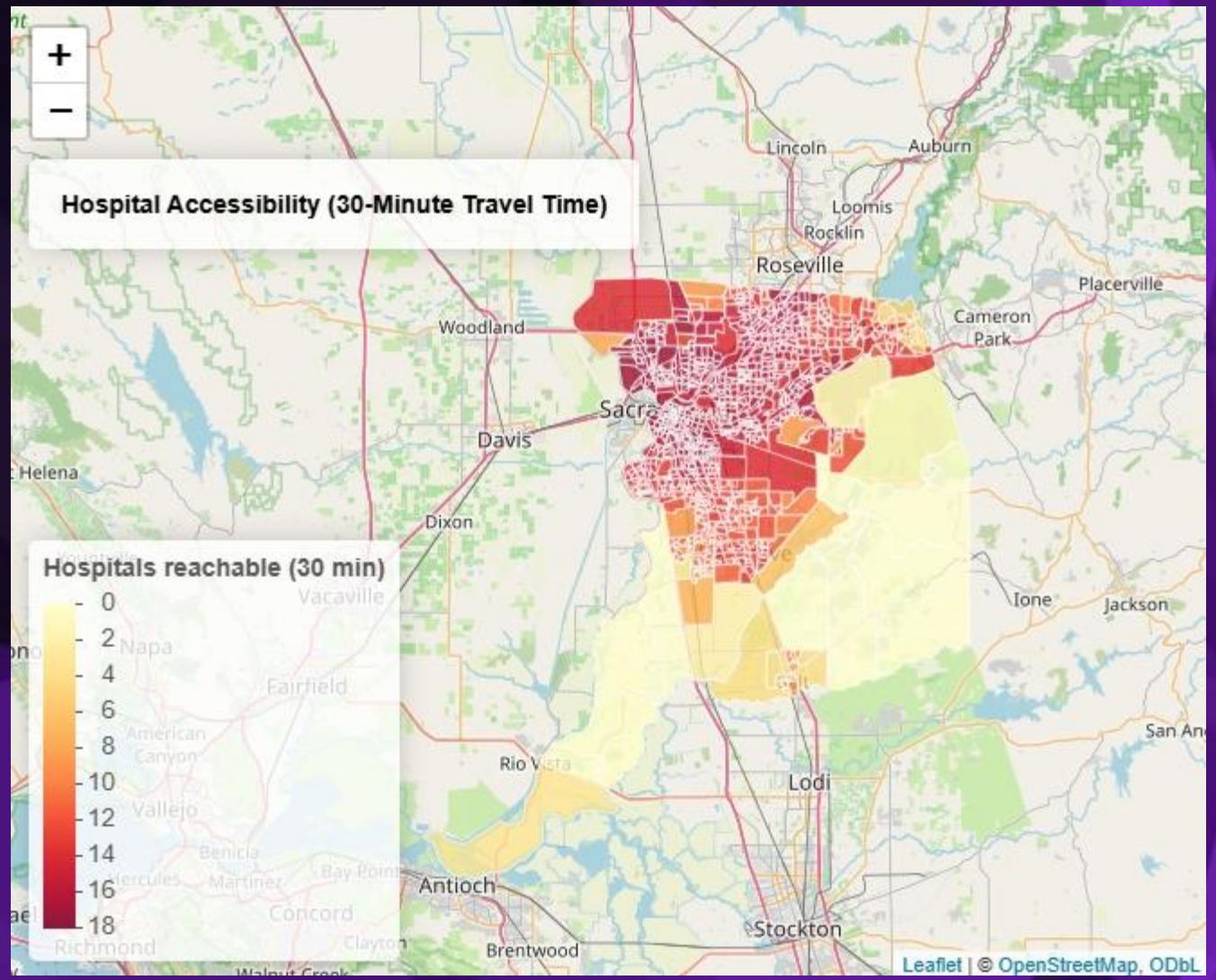
Moran I statistic standard deviate = 49.718, p-value < 2.2e-16
alternative hypothesis: greater
sample estimates:
Moran I statistic       Expectation          Variance
     0.8846039128     -0.0009775171      0.0003172765
```

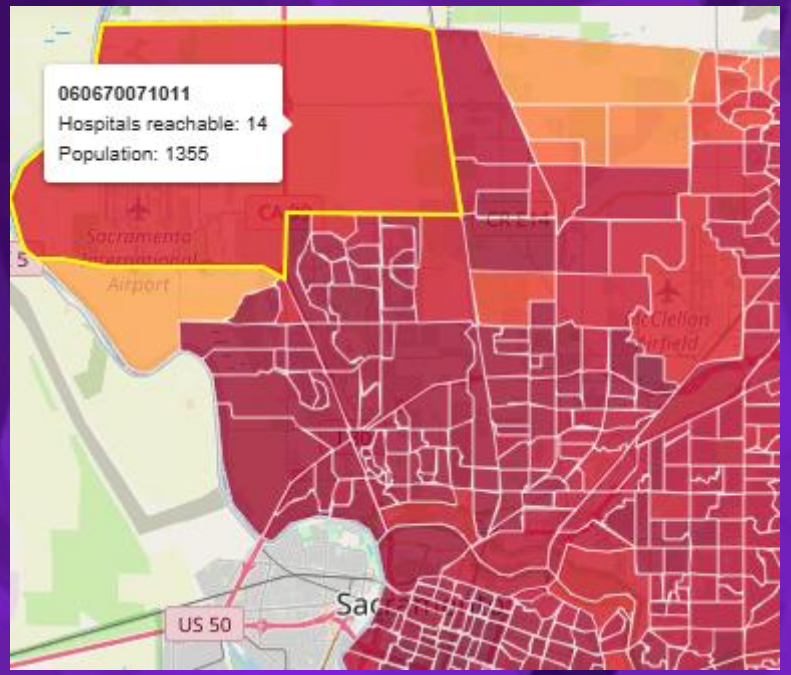
VISUALIZATION

LEAFLET

- Useful for creating detailed, high quality map visualizations within R
- Lots of customization options!
  - Examples: basemap, map view and zoom colors, sizes, text, etc.

```
hospital_access_map <- leaflet(block_groups) |>
  addControl(
    html = "<div style='            #Creating Map title
              font-size:20px;
              font-weight:bold;
              color:black;
              padding:10px;
              border-radius:6px;
              text-align:center;
            '>
              Hospital Accessibility (30-Minute Travel Time)
            </div>",
    position = "topleft"
  ) |>
  addTiles() |> #Setting basemap, lots of different options!
  setView(-121.5, 38.5, 9) |> #setting view and zoom level
  addPolygons(
    fillColor = ~pal(accessibility),
    fillOpacity = 0.75,
    color = "white",
    weight = 0.6,
    opacity = 1,
    label = labels,
    highlightOptions = highlightOptions(
      weight = 2,
      color = "#FFFF00",
      bringToFront = TRUE
    )
  ) |>
  addLegend(     #Adding legend
    pal = pal,
    values = ~accessibility,
    title = "Hospitals reachable (30 min)",
    opacity = 0.9,
    position = "bottomleft"
  )
hospital_access_map
```


Hospital Accessibility (30-Minute Travel Time)
Hospitals reachable (30 min)
0
2
4
6
8
10
12
14
16
18
Lincoln
Auburn
Loomis
Rocklin
Roseville
Placerville
Cameron Park
Woodland
Davis
Dixon
Vacaville
Napa
Fairfield
Vallejo
Benicia
Martinez
Concord
Antioch
Brentwood
Stockton
Lodi
Rio Vista
Ione
Jackson
Helena
Leaflet | © OpenStreetMap, ODbL

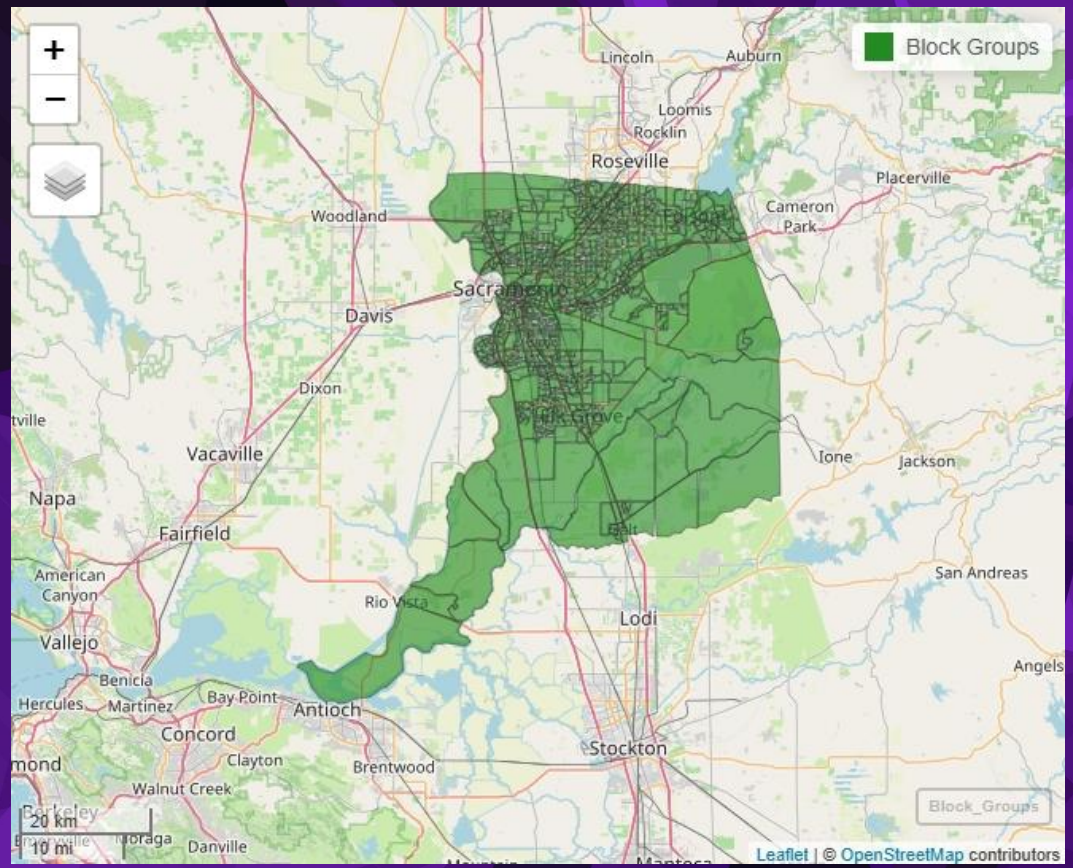

060670071011
Hospitals reachable: 14
Population: 1355
US 50

MAPVIEW

- An easier, simpler way to create maps in R
- Built on top of and acts as a wrapper for Leaflet
- I used this package to create the maps in this presentation!

```
#Importing library
library(mapview)

#Block Groups
bg_view <- mapview(block_groups, col.regions = "forestgreen",
                   layer.name = "Block Groups")
bg_view
```



CONCLUSION

- Analyzed spatial accessibility to hospitals within Sacramento Metropolitan Area
- Used a variety of open-source R packages for the workflow
  - Data acquitision
  - Data processing
  - Spatial statistics
  - Data visualization
- There are many more R packages out there that help with other GIS workflows



Check out the workflow for this talk on GitHub!